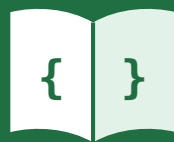

Module 09 — Distributed Data with Hazelcast

Exercises

Spring Boot Edu



2026-09-23

Contents

How to Work	2
Exercise 1 — A Client with a Near Cache (Easy)	2
Exercise 2 — Reserve a Whole Order (Hard)	3
Exercise 3 — Redis or Hazelcast? (Written Report)	3
Extra Challenge (Optional)	4

How to Work

1. The starter code is in `modules/09-hazelcast/exercise/`. Find the TODO comments.
2. The tests start their own embedded Hazelcast members. **No Docker is needed.**
3. Run the tests:

```
./mvnw -Pexercises -pl modules/09-hazelcast/exercise -am test
```

4. An exercise is done when all its tests are green. Exercise 3 is a written report and has no tests.
5. If you get stuck, read the hints first, then the solution in `modules/09-hazelcast/solution/`.

★ Tip

TestMembers (in the test sources) starts a member with its own cluster name, so the test classes never join each other.

Exercise 1 — A Client with a Near Cache (Easy)

Goal: make repeated reads of book data local to the client, and measure the difference.

The test starts a member and connects your client to it. ReadTimer is given: it reads the same key many times from the near-cached books map and from the orders map, which has no near cache.

Tasks (`exercisel.NearCacheClient.create`):

- TODO 1a — a near cache for the books map **only**, keeping deserialized objects.
- TODO 1b — local copies expire after 60 seconds and are invalidated when the entry changes in the cluster.
- TODO 1c — add the near cache to the client configuration.

Hints:

- Lesson section 3.6: `new NearCacheConfig("books"), setInMemoryFormat(InMemoryFormat.OBJECT), setTimeToLiveSeconds(...), setInvalidateOnChange(true).`
- `config.addNearCacheConfig(...).`
- The test prints the ReadTimer result. Compare the two numbers: how many times faster is the near cache on your machine?

Acceptance criteria: all 4 tests in `ExerciselTest` pass.

Estimated time: 20 minutes

Exercise 2 — Reserve a Whole Order (Hard)

Goal: an order with several books is reserved completely or not at all, while many orders run at the same time.

Tasks (`exercise2.OrderReservation.reserveAll`):

- TODO 2a — lock the key of every book in the order, in an order that avoids deadlocks.
- TODO 2b — if any book has less stock than ordered, change nothing and return `false`.
- TODO 2c — otherwise lower the stock of every book and return `true`.
- TODO 2d — release every lock you took, also when something goes wrong.

Hints:

- Lesson section 3.5: `stock.lock(isbn) / stock.unlock(isbn)` in `try/finally`.
- Check **all** books before you change **any** of them.
- Order A locks book 1 and waits for book 2. Order B locks book 2 and waits for book 1. Neither can continue. If everyone locks the books in the same order, e.g. sorted by ISBN, this cannot happen.
- Remember which keys you have already locked, so that finally unlocks exactly those.

Acceptance criteria: all 4 tests in `Exercise2Test` pass. `ordersThatNameTheSame-BooksInOppositeOrderDoNotDeadlock` fails after 20 seconds if your code can deadlock.

Estimated time: 45 minutes

Exercise 3 — Redis or Hazelcast? (Written Report)

Goal: choose the right tool for a requirement and justify the choice.

Write a report of 1–2 pages (`report.md` in your own notes; it is not checked in). Compare Redis (module 08) and Hazelcast (this module) for the Bookstore:

1. **Product detail cache** for 5 application instances: read 1,000 times per second, changed a few times per day.
2. **Stock reservation** during a campaign: many concurrent buyers for the same books.
3. **Sessions** of logged-in users.
4. **Price change notifications** to all instances.

For each case, answer:

- Which tool would you use, and in which topology (Redis server, embedded Hazelcast, Hazelcast client-server)?
- How does the data stay correct under concurrency (atomic commands, locks, entry processors)?
- What happens when one application instance or the data store restarts?
- What does it cost to run (extra servers, memory in every application, licences)?

Evaluation guide: a good report does not name one winner. It names at least one case where each tool fits better, and it mentions the licence question of the CP Subsystem.

Estimated time: 60 minutes

Extra Challenge (Optional)

Solve exercise 2 without any lock: write an `EntryProcessor` that reserves one book, and think about why this is not enough for an order with **several** books. Look up `IMap.executeOnKeys` and the Hazelcast transaction API (`TransactionContext`), and compare them with your lock-based solution.